

KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache

Zirui Liu^{*1} Jiayi Yuan^{*1} Hongye Jin² Shaochen (Henry) Zhong¹
Zhaozhuo Xu³ Vladimir Braverman¹ Beidi Chen⁴ Xia Hu¹

Abstract

Efficiently serving large language models (LLMs) requires batching of many requests to reduce the cost per request. Yet, with larger batch sizes and longer context lengths, the key-value (KV) cache, which stores attention keys and values to avoid re-computations, significantly increases memory demands and becomes the new bottleneck in speed and memory usage. Additionally, the loading of the KV cache causes the computational core to be idle, which limits the inference speed. A straightforward and effective solution to reduce KV cache size is quantization, which decreases the total bytes taken by KV cache. However, there is a lack of in-depth studies that explore the element distribution of KV cache to understand the hardness and limitation of KV cache quantization. To fill the gap, we conducted a comprehensive study on the element distribution in KV cache of popular LLMs. Our findings indicate that the key cache should be quantized per-channel, i.e., group elements along the channel dimension and quantize them together. In contrast, the value cache should be quantized per-token. From this analysis, we developed a tuning-free 2bit KV cache quantization algorithm named KIVI. With hardware-friendly implementation, KIVI can enable Llama, Falcon, and Mistral models to maintain almost the same quality while using $2.6\times$ less peak memory (including model weight). This reduction in memory usage enables up to $4\times$ larger batch size, bringing $2.35\times \sim 3.47\times$ throughput on real LLM inference workload. The source code is available at <https://github.com/jy-yuan/KIVI>.

^{*}Equal contribution. The order of authors is determined by flipping a coin. ¹Rice University ²Texas A&M University ³Stevens Institute of Technology ⁴Carnegie Mellon University. Correspondence to: Zirui Liu <zl105@rice.edu>, Jiayi Yuan <jy101@rice.edu>.

1. Introduction

Large Language Models (LLMs) have demonstrated strong performance across a wide range of tasks (Brown et al., 2020; Taylor et al., 2022; Yuan et al., 2023; Chuang et al., 2024). However, their deployment is very costly, requiring a large number of hardware accelerators such as GPUs. Given these substantial costs, one natural way to reduce the cost per request is to combine a sufficient number of requests together for batch processing. However, in this batch inference scenario, the key-value cache (KV cache), which holds the attention keys and values during generation to prevent re-computations, is becoming the new memory and speed bottleneck. This bottleneck becomes more pronounced with larger batch sizes and longer context lengths. For instance, in 540B PaLM, with a batch size of 512 and a context length of 2048, KV cache alone can take 3TB. This is 3 times the size of the model’s parameters (Pope et al., 2023). Also, the GPU SRAM has to load the whole KV cache from the GPU device memory for every token generated, during which the computational cores are idle. Thus, reducing KV cache size in LLMs while maintaining accuracy is important.

Existing works towards this problem can be roughly divided into three categories. First, some work suggests reducing the number of heads in KV cache, such as multi-query attention (Shazeer, 2019) and multi-group attention (Ainslie et al., 2023). However, these methods require either training the model from scratch or fine-tuning the existing model. Second, another research line reduces KV cache size by evicting unimportant tokens (Zhang et al., 2023). Third, some other works try to solve this problem from the system perspective, e.g., offloading KV cache (Sheng et al., 2023) or extending virtual memory and paging techniques into the attention mechanism (Kwon et al., 2023).

To reduce the size of KV cache, the most simple and effective way is to reduce the total bytes taken by KV cache, namely, quantization. Unlike the well-studied weight quantization (Lin et al., 2023; Xiao et al., 2023a; Zhao et al., 2024), to the best of our knowledge, only a few studies applied the vanilla 4bit round-to-nearest quantization to KV cache (Sheng et al., 2023; Zhang et al., 2023; Zhao et al., 2024) due to the streaming nature of KV cache or other complications. There is a lack of in-depth studies that ex-

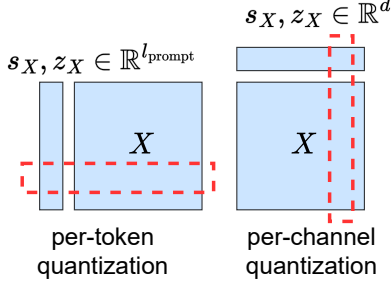


Figure 1: Definition of per-token and per-channel quantization. $X \in \mathbb{R}^{l_{\text{prompt}} \times d}$ is key/value cache where l_{prompt} is the number of tokens and d is the number of channels. z_X is the zero-point, s_X is the scaling factor.

Explore the element distribution of KV cache to understand the hardness and limitation of KV cache quantization. To fill the gap, we study the element distribution of KV cache. Our analysis suggests:

- For key cache, there are a few fixed channels whose magnitudes are very large, which is consistent with previous finding (Lin et al., 2023; Xiao et al., 2023a). Thus, as shown in Figure 1 right, key cache should be quantized per-channel, i.e., group elements along the channel dimension and quantize them together. In this way, it can confine the error to each individual channel, without impacting the other normal channels.
- For value cache, there is no obvious outlier pattern. Although value cache has no obvious outlier pattern, we experimentally show that it can only be quantized per-token because it is used to calculate the attention output, which is essentially a value cache mixer. As shown in Figure 1 left, the per-token quantization can confine the error inside each individual token and ensure that the quantization of one token does not adversely impact the others.

Based on the above insights, we propose KIVI, a plug-and-play extreme low-bit KV cache quantization method. KIVI quantizes key cache per-channel and quantizes value cache per-token. The per-token value cache quantization aligns well with the streaming nature of auto-regressive inference, allowing newly quantized tensors to be directly appended to the existing quantized value cache by token dimension. However, for per-channel key cache quantization, the quantization process spans different tokens, which cannot be directly implemented in this streaming setting. Since the number of tokens in key cache can be arbitrary, our key idea is to split key cache into two parts. The **first** part is the grouped key cache, which contains several groups of tokens and each group has a certain number of tokens. The **second** part is the residual key cache, which does not have a sufficient number of tokens to form a complete group. Similarly,

we split value cache into the grouped and residual parts to maintain the accuracy. We only apply group-wise quantization to the grouped key cache and value cache, while the residual key cache and value cache are kept in full precision. The grouped and residual parts can be combined using tiled matrix multiplication when computing attention scores. Our contributions are summarized as follows:

- **Extensive analysis regarding the outlier patterns and quantization error of KV cache in commonly-used LLMs.** Our observations suggest that key cache should be quantized per-channel and value cache should be quantized per-token. We also explain in depth why these caches require different quantization approaches.
- **A new plug-and-play 2bit KV cache quantization algorithm without any fine-tuning, KIVI, with hardware-friendly implementation.** We conduct an extensive evaluation for KIVI with Llama, Mistral, and Falcon on popular generation tasks. KIVI can efficiently compress KV cache to 2bit and bring $2.6\times$ peak memory usage reduction for Llama-2-7B, with little to no accuracy drop. With our efficient system implementation, this memory reduction, in return, enables up to $4\times$ larger batch size and brings $2.35\times \sim 3.47\times$ throughput.

2. Background: Attention Inference-Time Workflow

The LLM attention inference-time workflow involves two phases: i) the *prefill* phase, where the input prompt is used to generate KV cache for each transformer layer of LLMs; and ii) the *decoding* phase, where the model uses and updates KV cache to generate the next token, one at a time.

Prefill Phase. Let $X \in \mathbb{R}^{b \times l_{\text{prompt}} \times d}$ be the input tensor, where b is the batch size, l_{prompt} is the length of the input prompt, and d is the model hidden size. For convenience, we ignore the layer index here. The key, value tensors can be computed by

$$X_K = XW_K, X_V = XW_V,$$

where $W_K, W_V \in \mathbb{R}^{d \times d}$ are the key and value layer weight, respectively. After obtaining X_K and X_V , they are cached in the memory for the ease of decoding.

Decoding Phase. Let $t \in \mathbb{R}^{b \times 1 \times d}$ be the current input token embedding. Let $t_K = tW_K$ and $t_V = tW_V$ be the key and value layer output, respectively. We first update KV cache:

$$X_K \leftarrow \text{Concat}(X_K, t_K),$$

$$X_V \leftarrow \text{Concat}(X_V, t_V),$$

then calculate the attention output as:

$$\begin{aligned} t_Q &= tW_Q, \\ A &= \text{Softmax}(t_Q X_K^\top), \\ t_O &= AX_V, \end{aligned} \quad (1)$$

where W_Q is the weight matrix of the query layer. For ease of illustration, we ignore the attention output layer and the other parts of the inference workflow.

Memory and Speed Analysis. The above process is repeated until a special token indicating the sentence’s conclusion is reached. Let l_{gen} be the number of generated tokens. From the above analysis, the shape of KV cache is $b \times (l_{\text{prompt}} + l_{\text{gen}}) \times d$. To get a sense of the scale, consider the OPT-175B model with a batch size b 512, a prompt length l_{prompt} 512, and an output length l_{gen} 32. The KV cache requires 1.2TB, which is 3.8 times the model weights (Sheng et al., 2023). Besides the memory, the inference speed is also decided by the KV cache size. The GPU needs to load KV cache from GPU main memory to GPU SRAM once for every token generated during which the computational core of the chip is essentially idle (Pope et al., 2023; Kwon et al., 2023).

3. Methodology

In scenarios with long contexts or batched inferences, the memory and speed bottlenecks are storing and loading KV cache. The most simple and effective way to alleviate this problem is to reduce the total bytes occupied by KV cache, specifically, quantization. Following this motivation, we first evaluate the performance of the existing quantization method in Section 3.1. Our observations suggest that key and value cache should be quantized along different dimensions. We analyze the rationale behind this observation in Section 3.2. Then based on the analysis, we propose KIVI, a new KV cache quantization method along with its streaming data structure, detailed in Section 3.3.

3.1. Preliminary Study of KV Cache Quantization

As we analyzed in Section 2, KV cache functions as a streaming data structure, where the new tensor arrives sequentially. Thus, optimization-based methods like GPTQ (Frantar et al., 2022) are unsuitable for quantizing KV cache due to the overhead. To the best of our knowledge, the most flexible way for quantizing KV cache is the round-to-nearest quantization. The B -bit integer quantization-dequantization process can be expressed as:

$$Q(\mathbf{X}) = \lfloor \frac{\mathbf{X} - z_X}{s_X} \rfloor, \quad \mathbf{X}' = Q(\mathbf{X}) \cdot s_X + z_X,$$

where $z_X = \min \mathbf{X}$ is the zero-point, $s_X = (\max \mathbf{X} - \min \mathbf{X}) / (2^B - 1)$ is the scaling factor, and $\lfloor \cdot \rfloor$ is the rounding operation. Here we ignore the batch size for ease of

Table 1: The results of simulated KV cache group-wise quantization with various configurations. The group size is set as 32. $\mathbb{C}$ stands for per-channel quantization and $\mathbb{T}$ stands for per-token quantization. Please check the whole evaluation in Table 3.

Llama-2-13B	CoQA	TruthfulQA
16bit	66.37	29.53
4bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	66.48	29.51
2bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	52.93	24.98
2bit (K - $\mathbb{C}$, V - $\mathbb{C}$)	2.88	0.74
2bit (K - $\mathbb{T}$, V - $\mathbb{C}$)	2.80	0.26
2bit (K - $\mathbb{C}$, V - $\mathbb{T}$)	63.53	28.60

understanding. As shown in Figure 1, $\mathbf{X}$ is quantized along either the token or channel dimension group-wisely.

Considering the streaming nature of KV cache, previous studies often apply per-token quantization to both key and value cache since the newly quantized KV cache can be naively added to the existing quantized one along the token dimension (Sheng et al., 2023). While per-channel quantization is non-trivial, we have designed a padding method to implement per-channel quantization to explore its effect on both key and value cache.

Setting. In Table 1, we show the results of fake KV cache group-wise quantization with different configurations on the Llama-2-13B model for the CoQA and TruthfulQA tasks. We use a group size of 32 for all configurations. Here fake quantization means we simulate the quantization process by first quantizing KV cache into lower precision and then dequantizing it in the attention layer. For per-channel quantization, if the number of tokens is not divided evenly into groups, we add zero-padding to ensure it can be grouped perfectly. In this way, we ensure that all tokens in KV cache are quantized for a fair comparison. The detailed experimental setting can be found in Section 4.1. Specifically, we observe that:

OB 1. When using the commonly used per-token quantization to both key and value caches, INT4 precision can maintain accuracy. However, reducing it to INT2 results in a notable accuracy drop.

OB 2. When value cache is quantized per-channel, the accuracy significantly worsens regardless of how key cache is quantized.

OB 3. When using a lower numerical precision such as INT2, the most accurate approach is to quantize key cache per-channel and value cache per-token.

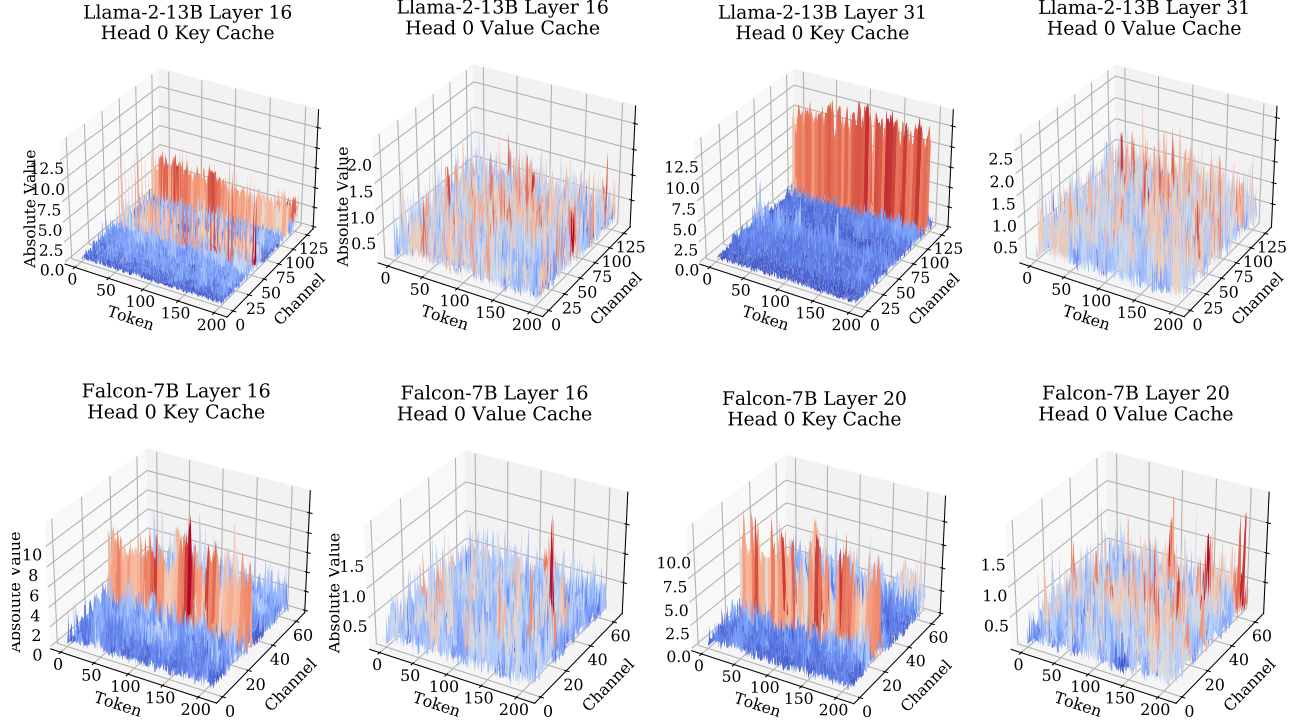


Figure 2: Magnitude of key and value cache for Llama-2-13B and Falcon-7B. We observe (1) for key cache, there are a few channels whose magnitudes are very large. (2) for value cache, there is no obvious outlier pattern.

3.2. Why Key and Value Cache Should Quantize Along Different Dimensions?

In Table 1, we observe that quantizing key cache per-channel and value cache per-token to 2bit results in a very small accuracy drop. Here we analyze why this configuration delivers better accuracy. In Figure 2 we visualize the original KV cache distribution at different layers. We observe that **in key cache, some fixed channels exhibit very large magnitudes, whereas in value cache, there is no significant pattern for outliers.**

Analysis of Key Cache. The above observation for key cache aligns with previous findings that certain fixed columns in activations exhibit larger outliers (Dettmers et al., 2022; Lin et al., 2023). The persistence of outliers within each channel means that per-channel quantization can confine the quantization error to each individual channel without impacting the other normal channels. Thus, Figure 2 explains why key cache should be quantized per-channel. In Table 2 we show key cache relative reconstruction error $\|\frac{\mathbf{X}_K - \mathbf{X}'_K}{\mathbf{X}_K}\|_F$, along with the relative attention score error $\|\frac{\mathbf{A} - \mathbf{A}'}{\mathbf{A}}\|_F$ where $\mathbf{A}' = \text{Softmax}(\mathbf{t}_Q \mathbf{X}'_K^\top)$. We observe that the per-token quantization can lead to almost $5\times$ larger attention score error than per-channel quantization, which is consistent with Figure 2.

Table 2: The relative error statistics averaged over all layers and all heads

Llama-2-13B	K Per-Token	K Per-Channel
Avg. $\ \frac{\mathbf{X}_K - \mathbf{X}'_K}{\mathbf{X}_K}\ _F$	13.67	4.55
Avg. $\ \frac{\mathbf{A} - \mathbf{A}'}{\mathbf{A}}\ _F$	47.00	9.60
Attention sparsity	84.3%	
	V Per-Token	V Per-Channel
Avg. $\ \frac{\mathbf{X}_V - \mathbf{X}'_V}{\mathbf{X}_V}\ _F$	4.57	3.73
Avg. Δ	3.55	49.89

Analysis of Value Cache. Unlike key cache, value cache does not show the channel-wise outlier pattern. Furthermore, Figure 2 alone cannot explain **OB2**, which indicates value cache should only be quantized per-token. This is because Figure 2 implies that errors should be comparable for both per-token and per-channel quantization, given the absence of a clear pattern. As shown in Equation (1), value cache is used to calculate the attention output $\mathbf{t}_O$. Instead of analyzing the quantization error of value cache $\mathbf{X}_V$, in Table 2 we analyze the relative error $\Delta = \|\frac{\mathbf{A}\mathbf{X}_V - \mathbf{A}\mathbf{X}'_V}{\mathbf{A}\mathbf{X}_V}\|_F$ with different quantization configurations. Surprisingly, we observe that the per-token quantization error is almost $15\times$